

Centro Universitário Filadélfia
Polo Salinópolis
Ciência de Dados e Inteligência Artificial

Aula 7

Qualidade de Código

Fundamentos de Propriedade Intelectual Para a Transferência de
Tecnologia de Software

Thiago Rafael da Silva Moura
trsmoura@yahoo.com.br

28 de novembro de 2025

CAPÍTULO 7

Qualidade de Código

Formatação, Linting e Verificação de Tipos
Engenharia de Software para Ciência de Dados

Roteiro da Aula

> 1. Estilo e Formatação

- > Por que a legibilidade importa?
- > PEP 8: O Padrão Ouro
- > Formataadores Automáticos: Black & isort

> 2. Linting (Qualidade)

- > Formataadores vs. Linters
- > Flake8 e Pylint
- > Complexidade Ciclomática

> 3. Verificação de Tipos

- > Tipagem Dinâmica vs. Estática
- > Type Hints (PEP 484)
- > MyPy: O Verificador Estático

> 4. Automação

- > Pre-commit hooks

Código é para Humanos

"Código é lido muito mais frequentemente do que é escrito." — Guido van Rossum.

Em Data Science, scripts tendem a crescer organicamente em notebooks. Sem padrões, eles se tornam ilegíveis e impossíveis de manter.



Por que formatar o código?



Carga Cognitiva

Formatação consistente permite que seu cérebro foque na **lógica**, não na "bagunça visual".



Colaboração

Evita discussões inúteis sobre "espaços vs. tabs" em Code Reviews. O time todo escreve como se fosse uma pessoa só.



Prevenção de Erros

Muitas vezes, má indentação em Python muda completamente a lógica do programa.

PEP 8: A Bíblia do Estilo

A **Python Enhancement Proposal 8** é o guia oficial de estilo para código Python.

Regras Fundamentais

- > Use **4 espaços** para indentação (nunca tabs).
- > Limite linhas a **79 caracteres** (ou 88/100 em times modernos).
- > Duas linhas em branco entre funções/classes de nível superior.
- > Codificação UTF-8.

```
# Correto (PEP 8) def calcular_media(x, y): retu
```

Espaçamento e Linhas

O espaço em branco é sintaxe em Python, mas também é respiro visual.

```
# Sim: Espaços ao redor de operadores i = i + 1 submetido += 1 x = x * 2 - 1 hipot = x*x + y*y # Exceção
```

Convenções de Nomes

Tipo	Convenção	Exemplo
Variáveis / Funções	snake_case	minha_variavel, calcular_media()
Classes	CamelCase (PascalCase)	RandomForestClassifier
Constantes	UPPER_CASE	MAX_ITERATIONS, PI
Privados (Convenção)	_leading_underscore	_metodo_interno()

O Dilema: Manual vs. Automático



Formatação Manual

Lenta, propensa a erros e gera debates intermináveis em Code Reviews. "Você esqueceu um espaço aqui".



Formatação Automática

Rápida, determinística e impessoal. O computador decide o estilo, os humanos discutem a arquitetura.

Ferramentas de Formatação

- > **autopep8**: Tenta consertar violações do PEP 8, mas é pouco agressivo. Mantém muito do estilo original do autor.
- > **Yapf (Google)**: Formata o código inteiro para o melhor estilo possível, altamente configurável.
- > **Black**: "The uncompromising code formatter". A ferramenta dominante hoje.

Black: O Formatador Intransigente

O Black toma todas as decisões de formatação por você.

Filosofia: O estilo do Black pode não ser o seu favorito, mas é consistente. Ao usá-lo, você ganha tempo e energia mental.

"Any color you like, as long as it's black." — Henry Ford



Black: Antes e Depois

Antes (Messy)

```
j = [1, 2, 3] def very_important_function(template: str, date=None, file_path=None, debug=False): pass
```

Organizando Imports (`isort`)

PEP 8 diz que imports devem ser ordenados: (1) Biblioteca Padrão, (2) Bibliotecas de Terceiros, (3) Aplicação Local.

```
# isort organiza automaticamente assim: import os import sys import numpy as np import pandas as pd from
```



Parte 2: Linting

Análise Estática de Qualidade

O que é um Linter?

Enquanto formatadores (Black) mudam a *aparência* do código, Linters analisam a **estrutura** em busca de erros lógicos, bugs potenciais e "code smells" (mau cheiro).

Eles avisam sobre variáveis não usadas, imports desnecessários e complexidade excessiva.



Flake8: O Linter Padrão

Combina três ferramentas em uma:

1. > **PyFlakes:** Checa erros (variáveis indefinidas, módulos não usados).
2. > **pycodestyle:** Checa estilo PEP 8.
3. > **McCabe:** Checa complexidade.

```
$ flake8 script.py script.py:4:1: F401 'os' imported but unused script.py:10:5: E225 missing white
```

Pylint: O Linter Rigoroso

O Pylint é muito mais detalhista que o Flake8. Ele dá uma "nota" (0 a 10) para o seu código.

Ele reclama de nomes de variáveis curtos demais (df, x), falta de docstrings, e muito mais.

```
# Saída do Pylint C0111: Missing module docstring
```

Complexidade Ciclomática

É uma métrica que conta o número de "caminhos independentes" no seu código ('if', 'for', 'while', 'except').

Baixa (1-10)

Código simples, fácil de testar.

Média (11-20)

Risco moderado, considere refatorar.

Alta (>20)

Código "espaguete". Difícil de manter e testar. **Refatore!**

Quando o Linter erra (`noqa`)

Às vezes, você tem um bom motivo para violar uma regra. Você pode silenciar o linter explicitamente.

```
import modulo_nao_usado # noqa: F401 password = "123456" # noqa (Ignora tudo nessa linha)
```

Use com moderação! Um código cheio de `noqa` é um sinal ruim.



Parte 3: Verificação de Tipos

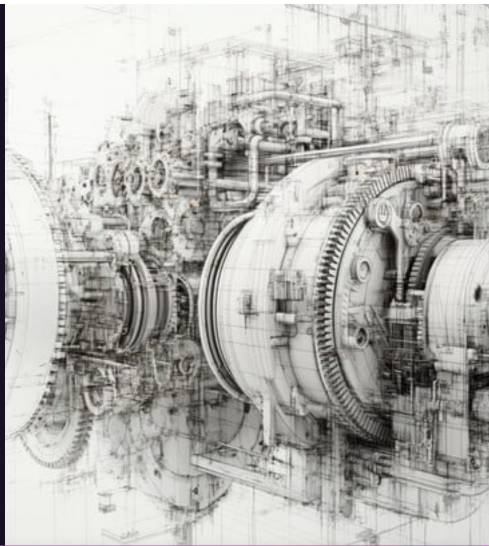
Type Hinting e MyPy

Dinâmico vs. Estático

Python é **dinamicamente tipado**. Você não precisa declarar se `x` é inteiro ou string. Isso é ótimo para prototipagem rápida.

Porém, em sistemas grandes, isso causa bugs de "TypeError" que só aparecem quando o código roda.

Type Hints (PEP 484) trazem o melhor dos dois mundos.



Sintaxe de Type Hinting

Adicionamos anotações após o nome da variável ou argumento. O Python ignora isso na execução, é apenas "documentação".

```
# Variáveis idade: int = 25 nome: str = "Alice" # Funções (Argumentos → Retorno) def saudacao(nome: str):
```

Tipos Complexos

Para listas, dicionários e tuplas, definimos o conteúdo interno.

```
# Python 3.9+ (Built-in collections) notas: list[float] = [9.5, 8.0, 7.5] usuario: dict[str, int] = {"i
```

Optional e Union

Union

Quando uma variável pode ser de múltiplos tipos.

```
def processar(x: int | str): # x pode ser int OU str
```

Optional

Quando o valor pode ser do tipo X ou None.

```
from typing import Optional def buscar_user(id) → Optional
```

MyPy: O Juiz

As dicas de tipo são inúteis sozinhas. Precisamos de uma ferramenta para verificá-las.

O **MyPy** analisa seu código *estaticamente* (sem rodar) e encontra violações de tipo.

Escreva Código

Com Type Hints

Rode MyPy

```
$ mypy script.py
```

Corrija Erros

Antes de ir para produção

MyPy em Ação

```
def soma(a: int, b: int) → int: return a + b # Erro sutil que o MyPy pega: resultado = soma(10, "20")
```

```
$ mypy script.py script.py:5: error: Argument 2 to "soma" has incompatible type "str"; expected "int"
```

Prós e Contras da Tipagem

Vantagens

- > Documentação viva (nunca desatualiza).
- > IDEs ficam mais inteligentes (autocomplete).
- > Pega bugs antes da execução (Refatoração segura).

Desvantagens

- > Mais código para escrever.
- > Curva de aprendizado inicial.
- > Pode ser verboso em estruturas de dados complexas.

Automação

Não confie na memória humana para rodar Black, Flake8 e MyPy antes de cada commit.

Automatize o processo para garantir que nenhum código ruim entre no repositório.



Pre-commit Hooks

Framework que roda verificações automaticamente quando você tenta fazer `git commit`.

Arquivo `.pre-commit-config.yaml`:

```
repos: - repo: https://github.com/psf/black rev: 23.3.0 hooks: - id: black - repo: https://github.com/pycqa/flake8 rev: 6.0.0 hooks: - id: flake8
```

Se o Black mudar algo ou o Flake8 achar erro, o commit é bloqueado!

O Stack de Qualidade Moderna

Formatação

Black

Estilo consistente.

Imports

isort

Organização.

Linting

Flake8

Bugs e PEP 8.

Tipagem

MyPy

Segurança de tipos.

Tudo orquestrado via **pre-commit**.



Dúvidas?

Obrigado!